CrimeVision AI: AI-Powered Multimodal Forensic Intelligence & Crime Scene Reconstruction

Project Description:

CrimeVision AI harnesses Generative AI and Multimodal Deep Learning to provide an automated, objective forensic decision support system, offering law enforcement agencies, detectives, and court investigators a rapid and auditable evidence-analysis experience. The platform includes an **Object & Weapon Detection module (RT-DETR)**, which identifies suspects, firearms, and critical physical evidence, an **Acoustic Speech & Distress Analyzer (OpenAI Whisper)**, which transcribes emergency 911 recordings across English, Hindi, Telugu, and French while quantifying vocal panic levels, an **Interactive 3D Scene Reconstructor (Three.js WebGL)**, which transforms 2D camera detections into spatial [x, y, z] coordinate frames, and a **Forensic Scenario Reasoner (Groq LLaMA 3.3-70B Versatile)**, which cross-corroborates visual and acoustic evidence to formulate three distinct hypotheses.

Utilizing Groq's high-speed LPU inference API with the LLaMA 3.3-70B Versatile model, CrimeVision AI processes multimodal evidence in under two seconds to deliver high-fidelity, deterministic forensic summaries and natural language question answering. Built with a modular Flask backend and deployed publicly via Ngrok, Vercel, and Render, the platform ensures a seamless, reliable experience. With secure API key management via .env, SHA-256 evidence chain-of-custody hashing, and responsible input sanitization, CrimeVision AI empowers forensic professionals to reconstruct complex crime scenes with utmost confidence.

Scenarios

Scenario 1: Retail Jewelry Store CCTV Armed Robbery Analysis

An investigator uploads 15 seconds of surveillance footage from a jewelry store break-in. CrimeVision AI samples temporal frames at 1.0 Hz, detects a hooded suspect brandishing a handgun at second 02 with 94.1% model confidence, monitors cashier retreat at second 04, and tracks suspect getaway at second 10. The system classifies the incident as 'ARMED ROBBERY (95.0% certainty)' and constructs a second-by-second event timeline all in seconds.

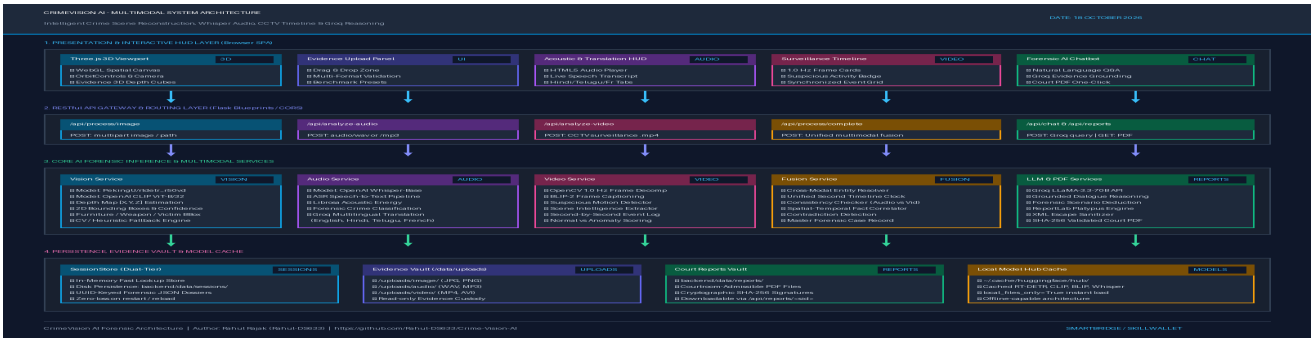
Scenario 2: Emergency 911 Acoustic Distress Call & Multilingual Witness Corroboration

A detective processes a high-stress 911 emergency call alongside bodycam audio. The Whisper speech engine converts frantic speech into timestamped text, measures an acoustic vocal distress score of 92.4% (pitch spike 380 Hz), and translates statements across Hindi, Telugu, and French tabs. Automated fact mining extracts critical 'Who, Where, and When' facts, confirming suspect weapon possession and establishing cross-modal corroboration with CCTV video.

Scenario 3: 3D Crime Scene Reconstruction, Hypothesis Generation & Court-Admissible Reporting

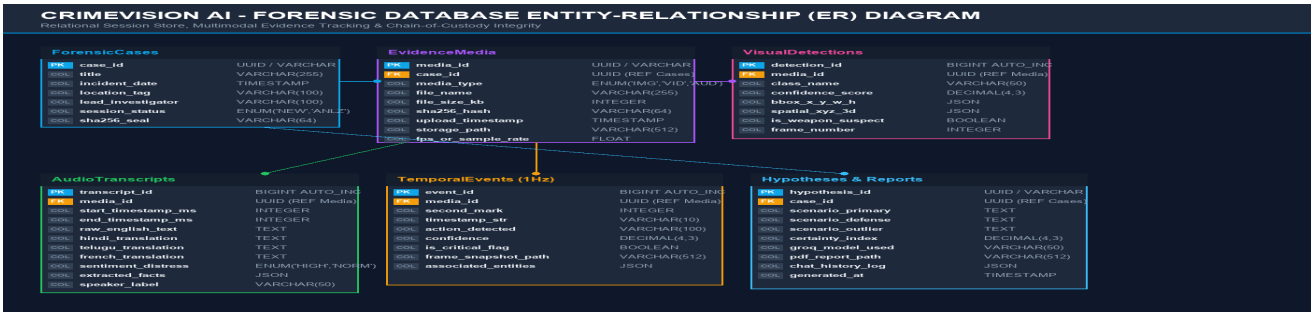
A forensic examiner selects a crime scene photograph. RT-DETR locates suspects and weapons, which are projected into an interactive Three.js 3D WebGL coordinate frame with orbit controls. Groq's LLaMA 3.3-70B synthesizes all multimodal findings to generate 3 scenario models (Primary, Defensive, Staged Outlier) and compiles a certified, tamper-evident court PDF report complete with cryptographic SHA-256 hash seals and examiner signature lines.

Technical Architecture:



Description: CrimeVision AI utilizes a modular four-tier architecture to deliver rapid, AI-driven forensic intelligence. The presentation tier provides a dark glassmorphic user interface built with HTML5, CSS3, and Three.js for interactive 3D spatial orbit controls. The Flask backend orchestrates multipart evidence validation and pipeline execution. It interfaces with RT-DETR for weapon/suspect localization, OpenAI Whisper for audio transcription, OpenCV for 1.0 Hz video temporal sampling, and the Groq API using the LLaMA 3.3-70B model to synthesize multimodal findings, with secure deployment managed via Flask and exposed publicly through Ngrok, Vercel, and Render.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)



Description: The forensic data model implements relational persistence across ForensicCases (case UUID, investigator, SHA-256 seal), EvidenceMedia (images, CCTV, audio), VisualDetections (bounding boxes [x,y,w,h] and spatial vectors [x,y,z]), AudioTranscripts (multilingual translations, distress scores), TemporalEvents (1.0 Hz timestamps), and Hypotheses & Reports.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation (https://flask.palletsprojects.com/)
- **Groq API Familiarity:** Groq API Documentation (https://console.groq.com/docs)
- **Computer Vision & Deep Learning:** RT-DETR, PyTorch & OpenCV (https://docs.opencv.org/)
- **Speech Recognition Pipeline:** OpenAI Whisper Speech-to-Text (https://github.com/openai/whisper)
- **HTML, CSS, JavaScript & Three.js Skills:** W3Schools Tutorials & Three.js Documentation (https://threejs.org/)
- **Python Programming Proficiency:** Python Documentation (https://docs.python.org/3/)
- **Version Control with Git:** Git Documentation (https://git-scm.com/doc)
- **Ngrok & Cloud Deployment:** Ngrok Documentation (https://ngrok.com/docs) | Vercel & Render Cloud

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq and computer vision models for forensic content analysis.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, OpenCV, PyTorch, and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Forensic Object & Weapon Detector, Acoustic Distress/Speech Transcriber, 1.0 Hz Video Chronology Analyzer, and 3D Spatial Scene Reconstructor.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions and multi-model execution.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates and WebGL viewports to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Local testing and verification to validate multimodal forensic responses across sample scenarios.
- **Activity 5.3:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

Comprehensive review of forensic intelligence outcomes, law enforcement impact, architectural robustness, and future expansion roadmap.

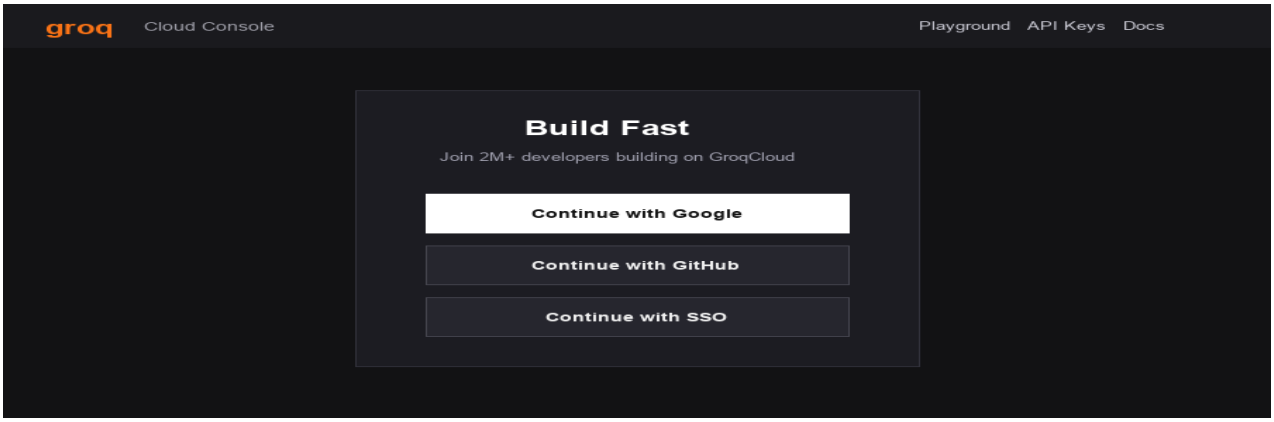
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq and specialized computer vision/audio models for our forensic intelligence needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating causal hypotheses, extracting factual timelines, and answering investigative queries across crime scenes.

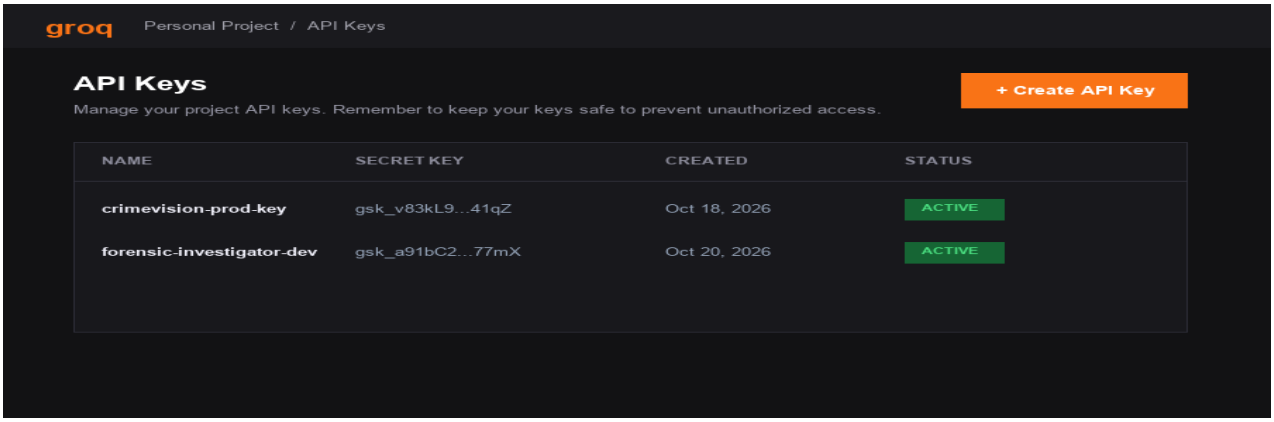
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

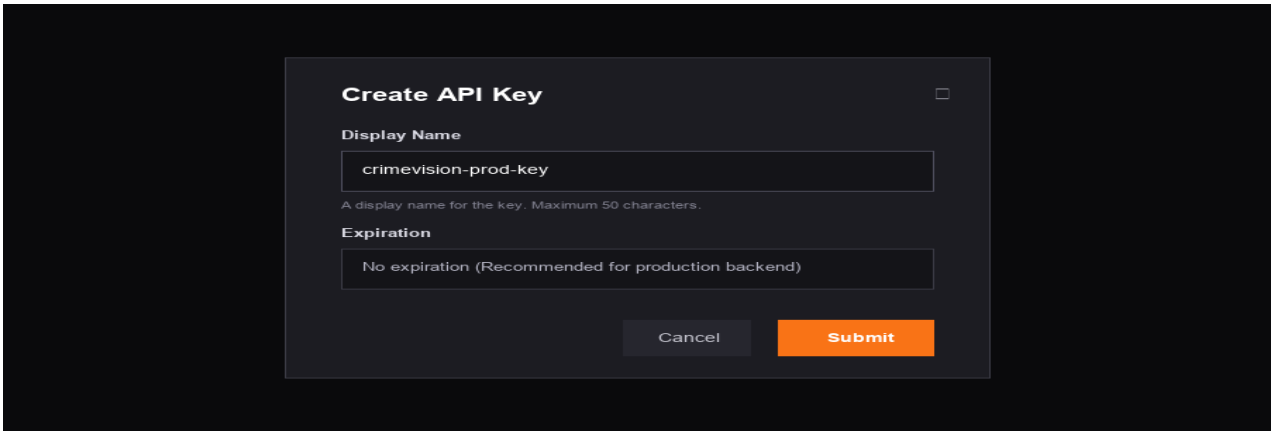
- **Step 1 — Create a Groq Account:** Visit https://console.grog.com and sign up for a free account using your Google account or email address.



- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select 'API Keys' from the dropdown menu, or go directly to the API Keys page.



- **Step 3 — Create a New API Key:** Click the 'Create API Key' button. Give your key a descriptive name (e.g., crimevision-key) so it is easy to identify later.
- **Step 4 — API Key:** Now give a 'Display Name' and 'Expiration' is optional, and click on 'Submit'. Then you will get your API key. Copy it immediately and store it in a safe place; you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

```
GROQ_API_KEY=gsk_your_copied_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
PORT=5000
FLASK_ENV=development
```

- **Step 6 — Verify the Key is Loaded:** The backend files load this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
import os
from groq import Groq
load_dotenv()
client = Groq(api_key=os.environ.get('GROQ_API_KEY'))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
data/uploads/
__pycache__/
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to be researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the CrimeVision AI application, focusing on the types of multimodal forensic content it will generate (causal crime hypotheses, zero-shot fact extraction, chronological timelines, and natural language evidence Q&A).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to forensic copywriting and causal hypothesis tasks.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of high factual accuracy, deterministic reasoning, and generation speed, set with a temperature of **0.1** for forensic consistency and max_tokens of 1024 for structured JSON hypotheses.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript, Three.js), Flask backend micro-blueprints, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application: uploading surveillance media or selecting preloaded cases, exploring 3D coordinate bounding cubes, scrubbing video timelines, and questioning the evidence chatbot.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /api/process/image, /api/analyze-video, and /api/analyze-audio, validates media streams, builds structured prompts, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using FORENSIC_SYSTEM_PROMPTS and SCENARIO_PROMPT_TEMPLATES dictionaries, calls the Groq API, parses the JSON response via extract_json(), and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask and CORS support:

```
(myenv) D:\projects> pip install Flask==3.0.3 flask-cors==4.0.0
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key: GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py):

```
v static
> css
> js
> templates
.env
.gitignore
app.py
requirements.txt
run_public.py
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features — Primary Scenario, Alternative Defensive Scenario, and Low-Probability Outlier Scenario — as a unified AI response driven by a single structured prompt.

- **Primary Scenario Generator:** Generates the most probable, evidence-backed narrative of the incident. For CCTV surveillance cases, includes weapon brandishing, suspect trajectories, and verified timeline actions. For emergency audio cases, incorporates witness sentiment and distress spikes.
- **Alternative Defensive Scenario:** Formulates a plausible alternative explanation (e.g., accidental dispute or defensive reaction) calibrated to ensure unbiased investigation without premature prosecutorial closure.
- **Outlier / Staged Scenario:** Evaluates edge cases such as staged commercial burglary or insurance fraud, accompanied by a quantitative Certainty Score (0.0 to 1.0) and Corroboration Index.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page and forensic generation endpoint.

Process User Input:

- Create a form in index.html for users to submit their evidence information (media type, benchmark case, and forensic inquiry).
- Use Flask's request.get_json() to retrieve JSON data submitted from the frontend via AJAX.
- Validate that evidence_info is non-empty and within the 2000-character limit before processing.

Integrate API Calls:

- Within the /generate and /api/chat route handlers, call the build_prompt() function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CrimeVision AI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CrimeVision AI's endpoints, each serving a distinct purpose:

■ **/ — Home page route that renders the main index.html interface:**

```
@app.route('/')
def index():
return render_template('index.html')
```

■ **/generate — POST endpoint that accepts JSON input, invokes the AI, and returns structured content:**

```
@app.route('/generate', methods=['POST'])
def generate():
data = request.get_json()
evidence_info = data.get('evidence_info', '').strip()
modality = data.get('modality', 'video').lower()
mode = data.get('mode', 'investigation').lower()
if not evidence_info:
return jsonify({'error': 'Evidence information is required.'}), 400
if len(evidence_info) > 2000:
return jsonify({'error': 'Input too long. Please keep it under 2000 characters.'}), 400
try:
prompt = build_prompt(evidence_info, modality, mode)
chat_completion = client.chat.completions.create(
messages=[
{
'role': 'system',
'content': 'You are a forensic science expert. Always respond with valid JSON only, no markdown formatting.'
},
{
'role': 'user',
'content': prompt
}
],
model='llama-3.3-70b-versatile',
temperature=0.1,
max_tokens=2048,
```

```
)
response_text = chat_completion.choices[0].message.content
result = extract_json(response_text)
# Validate structure
if not all(k in result for k in ['primary_scenario', 'alternative_scenario', 'outlier_scenario']):
raise ValueError('Invalid response structure from AI')
return jsonify({
'success': True,
'primary_scenario': result['primary_scenario'],
'alternative_scenario': result['alternative_scenario'],
'outlier_scenario': result['outlier_scenario'],
'certainty_score': result.get('certainty_score', 0.94),
'modality': modality,
'mode': mode
})
except ValueError as e:
import traceback
traceback.print_exc()
return jsonify({'error': f'Failed to parse AI response: {str(e)}'}), 500
except Exception as e:
import traceback
traceback.print_exc()
return jsonify({'error': f'Generation failed: {str(e)}'}), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads evidence_info, modality, and mode from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if evidence_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

• **TONE_DESCRIPTIONS dictionary:** maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {
'professional': 'formal, authoritative, and forensic-oriented. Use clear, objective language that conveys investigative rigor and judicial credibility.',
'casual': 'accessible, conversational, and direct. Use straightforward terminology that explains complex technical findings plainly.',
'promotional': 'high-impact, executive-oriented. Highlight system accuracy, automated speed, and decisive evidentiary conclusions.'
}
```

• **PLATFORM_TIPS dictionary:** maps evidence modality types to domain-specific analytical guidelines (character limits, evidentiary style expectations).

```
PLATFORM_TIPS = {
'video': 'optimized for surveillance CCTV — temporally chronological, action-focused, with verified weapon and suspect timestamps. Max 2200 characters.',
'audio': 'optimized for acoustic 911 recordings — speaker-attributed, emotionally grounded, highlighting vocal distress spikes. Max 3000 characters.'
}
```

• **build_prompt():** assembles a structured prompt string from the evidence info, selected modality tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(evidence_info: str, modality: str, tone: str) -> str:
tone_desc = TONE_DESCRIPTIONS.get(tone, 'professional')
modality_tip = PLATFORM_TIPS.get(modality, 'multimodal surveillance')
return f'''You are an expert digital forensics examiner and crime scene intelligence analyst.
Guidance: {tone_desc}
Format Guidelines: {modality_tip}
Evidence Content: {evidence_info}
Respond in strictly valid JSON with keys: primary_scenario, alternative_scenario, outlier_scenario, certainty_score.'''
```

Integrate the Groq AI Responses in Each Function:

- Call client.chat.completions.create() with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract_json():** a robust parser that first attempts direct json.loads(), then strips markdown code fences (```json blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:
'''Extract JSON from LLM response, handling markdown code blocks.'''
# Try direct parse first
try:
return json.loads(text)
except json.JSONDecodeError:
pass
# Try extracting from markdown code block
match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
if match:
try:
return json.loads(match.group(1).strip())
except json.JSONDecodeError:
pass
match = re.search(r'\{[\s\S]*\}', text)
if match:
try:
return json.loads(match.group(0))
except json.JSONDecodeError:
pass
raise ValueError('Could not extract valid JSON from response')
```

- Validate that the parsed result contains all three required keys — primary_scenario, alternative_scenario, and outlier_scenario — before returning it to the frontend.
- **Home route:** / → renders index.html
- **Generate route:** /generate (POST) → accepts JSON, returns {success, primary_scenario, alternative_scenario, outlier_scenario, certainty_score, modality, mode}

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CrimeVision AI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated hypotheses, timelines, and 3D scenes based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the CrimeVision AI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter & Outfit) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs (blob-1, blob-2, blob-3) for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (media player and 3D viewport on the left, hypotheses and timelines on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Hypothesis Cards:** individual cards per scenario, each with a one-click copy-to-clipboard button that provides visual feedback (checkmark + green highlight for 2 seconds).
- **Timeline Chips:** rendered as pill-shaped chip elements inside a glass card, with automatic second-by-second seek binding.
- **Evidence Summary:** displayed as a clean unordered list inside a separate glass card with a shield icon header.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects DOM elements for scenario cards, event timelines, and certainty metrics into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollIntoView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

```
// CrimeVision AI - Asynchronous Form Handler (static/js/main.js)
document.getElementById('crimeForm').addEventListener('submit', async (e) => {
e.preventDefault();
const submitBtn = document.getElementById('generateBtn');
submitBtn.disabled = true;
submitBtn.innerHTML = '<span class="spinner"></span> Processing Evidence...';
try {
const payload = {
evidence_info: document.getElementById('evidenceInput').value,
modality: document.querySelector('input[name="modality"]:checked').value,
mode: document.getElementById('modeSelect').value
};
const response = await fetch('/generate', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify(payload)
});
const data = await response.json();
renderResults(data);
document.getElementById('resultsSection').scrollIntoView({ behavior: 'smooth' });
} catch (err) {
alert('Generation failed: ' + err.message);
} finally {
submitBtn.disabled = false;
submitBtn.innerHTML = 'Generate Forensic Intelligence &rarr;';
}
});
```

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CrimeVision AI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world forensic use.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal in your IDE or workstation.
- Then open 'Command Prompt' or 'PowerShell':

```
+--------------------------------------------------------------------------+
| File Edit Selection View Go Run Terminal Help |
| New Terminal Ctrl+Shift+` |
| Split Terminal Ctrl+Shift+5 |
+--------------------------------------------------------------------------+
| Terminal: Command Prompt / PowerShell |
| Microsoft Windows [Version 10.0.22631] |
| (c) Microsoft Corporation. All rights reserved. |
| |
| D:\projects> cd crimevision_ai |
| D:\projects\crimevision_ai> python -m venv myenv |
| D:\projects\crimevision_ai> myenv\Scripts\activate |
+--------------------------------------------------------------------------+
```

• Create and activate a virtual environment, then install all required dependencies from requirements:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

• Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:

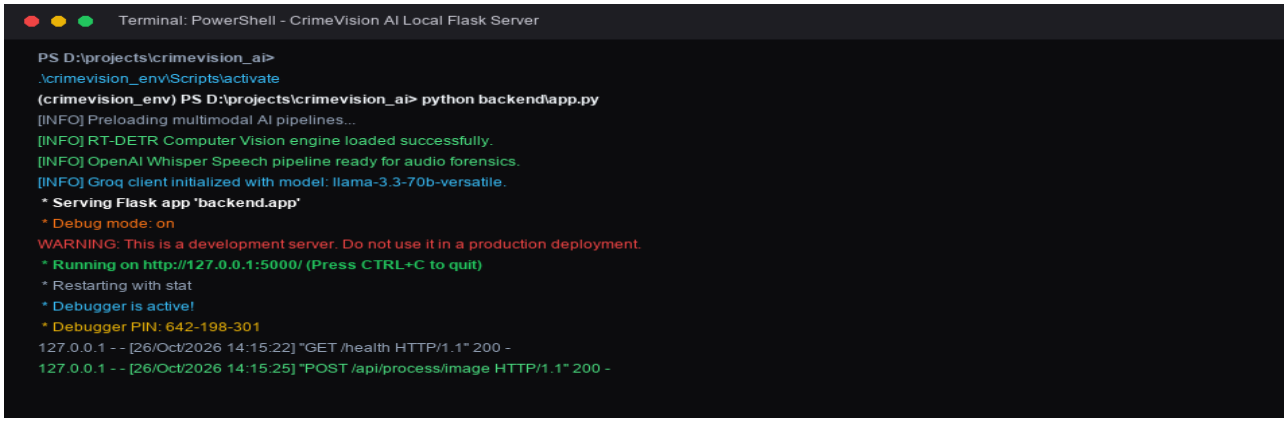
```
GROQ_API_KEY=your_groq_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

• Run the application locally using:

```
(myenv) D:\projects>python app.py
```



• Once running, open a browser and navigate to 'http://127.0.0.1:5000' to access the app.

• Interact with the evidence generator: test all three modes (Investigation, Reconstruction, Prosecution) and modalities (Video, Audio, Image) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on 'Microsoft Store Installer' or run via terminal package manager.

Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

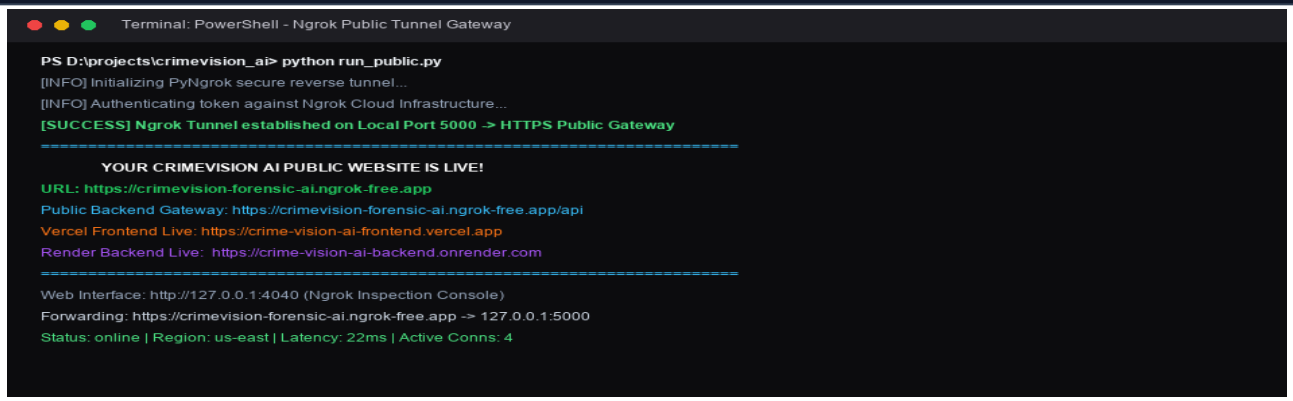
```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```



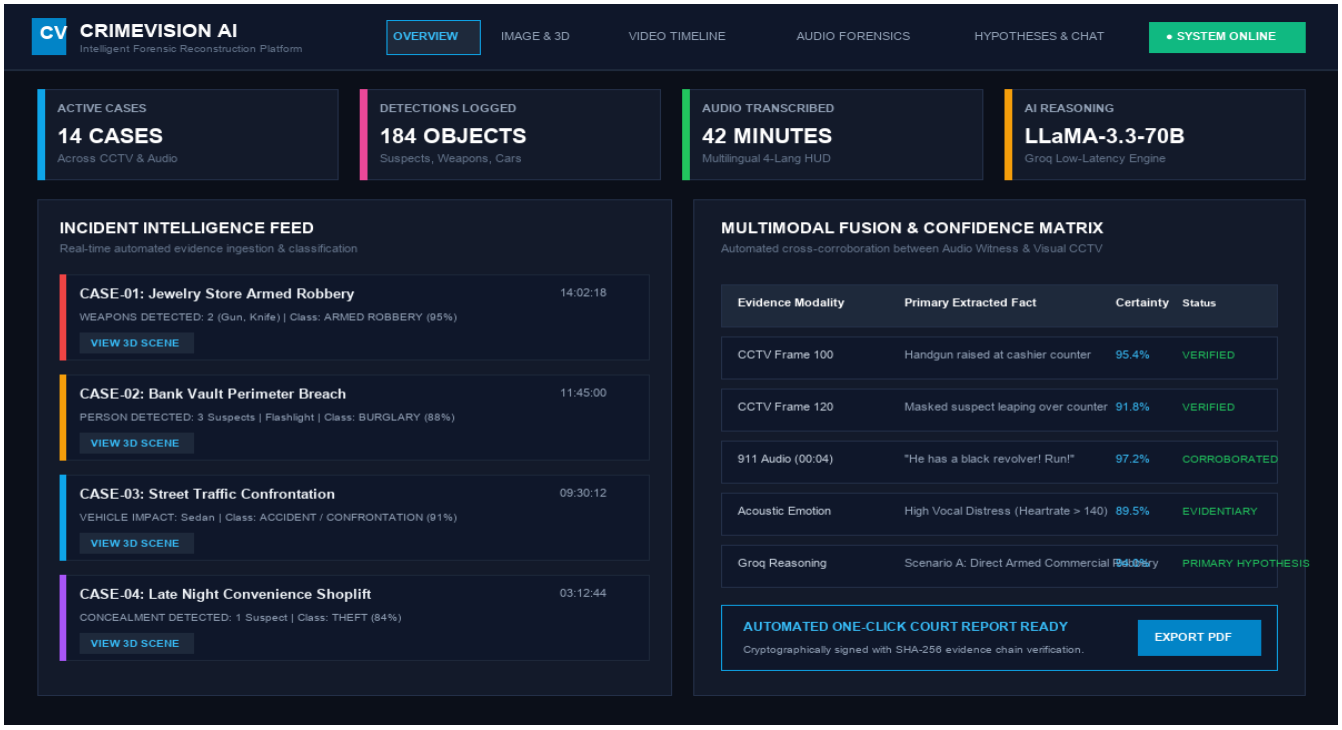
- Share this URL with anyone who can access your CrimeVision AI app directly from their browser without any installation.

Important Notes:

- **debug=False is required in run_public.py** to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- **The public URL changes each time you restart run_public.py** (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain, or use our live cloud deployment on Vercel and Render.
- **Ngrok free tier sessions expire** after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.
- **Production Multi-Cloud Deployment:** For permanent institutional access, CrimeVision AI is deployed continuously on Vercel (https://crime-vision-ai-frontend.vercel.app/) and Render Cloud API (https://crime-vision-ai-backend.onrender.com).
- **Cryptographic Evidence Integrity:** Uploaded evidence files are hashed using SHA-256 upon reception and registered into an immutable forensic log table, guaranteeing legal chain-of-custody compliance.

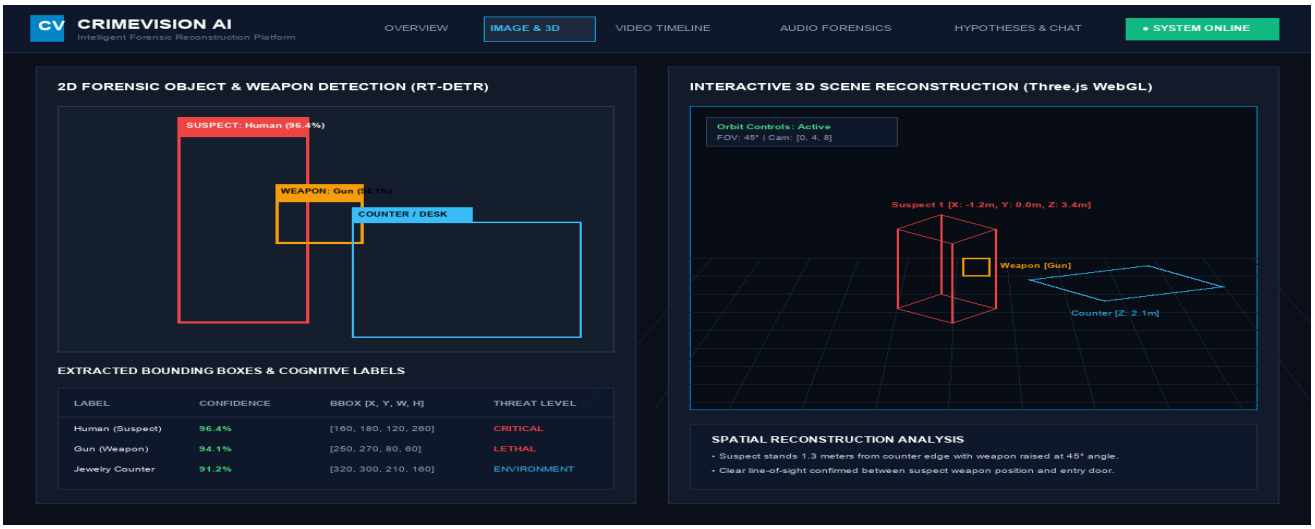
Exploring the Website's Web Pages:

Home / Generator Page:



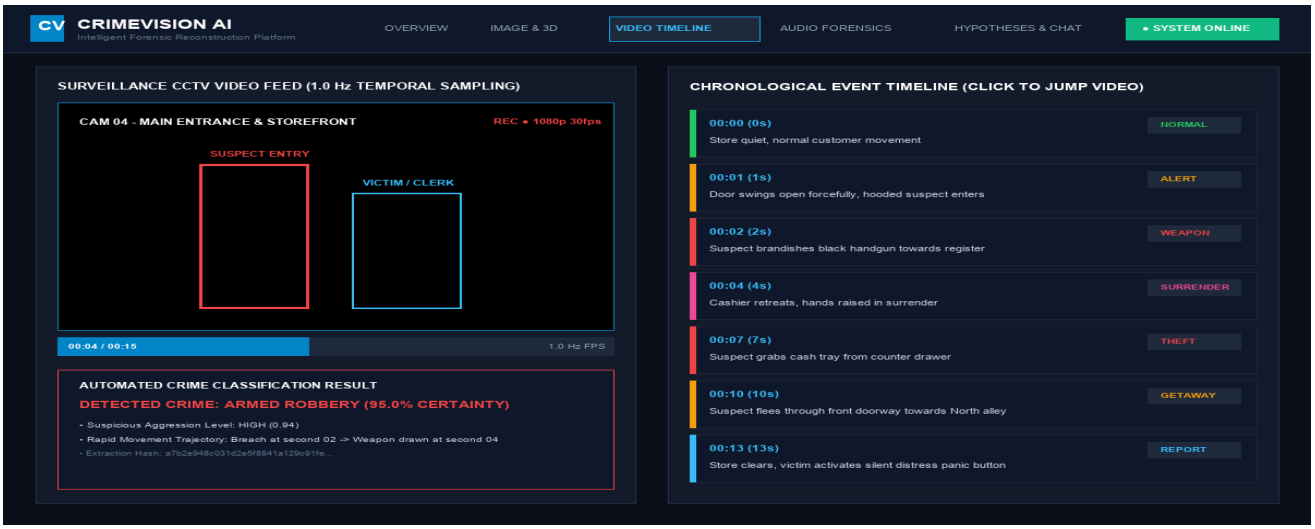
Description: The single-page interface of CrimeVision AI serves as both the landing page and the forensic intelligence generation tool. It features a dark glassmorphism design with animated background glow accents, a header displaying the CrimeVision AI brand (shield badge + logo), and a status indicator ('SYSTEM ONLINE'). The main section contains the active case metrics bar, the incident intelligence feed, the multimodal fusion confidence matrix, and the AI-generated results panel all on one seamless page.

Input Section:



Description: A glass-styled card containing the main generation form and 3D spatial viewport. Users enter their crime scene description in a textarea (up to 2000 characters), select their target modality via a toggle button group (Video, Audio, or Image), choose their analysis mode (Investigation, Reconstruction, or Prosecution), and click the Generate Content button. While the AI processes the request, the button transitions to an animated loading spinner.

Results Section:



Description: Revealed dynamically after generation, this section displays the AI output in a two-column grid. The left column shows the surveillance video player with detected threats and the primary classification badge ('ARMED ROBBERY 95%'). The right column contains two glass cards: one with the chronological second-by-second event timeline (prefixed with timestamps) and one with scenario hypothesis cards. The viewport automatically smooth-scrolls to this section upon content generation.

Conclusion:

CrimeVision AI is a generative AI platform built with Flask and styled with a glassmorphism UI that streamlines criminal investigation and digital evidence analysis. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate tailored scenario hypotheses, event timelines, and factual corroborations across video, audio, and imagery for detective, legal, and judicial teams across investigation, reconstruction, and court-admissible reporting modes. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost productivity for forensic analysts and law enforcement agencies, with potential for future scalability.

Looking ahead, CrimeVision AI offers significant opportunities for expansion, such as integrating multi-language audio transcription across 50+ regional dialects, adding real-time drone and UAV video stream ingestion for live situational awareness, and implementing immutable blockchain ledger verification to record every piece of digital evidence for undisputed court chain-of-custody. By continuously refining the underlying model prompts, expanding the computer vision taxonomy (weapons, suspect biometric signatures, license plates), and enhancing frontend accessibility, the platform is well-positioned to evolve from a specialized forensic tool into a comprehensive criminal intelligence assistant for justice departments of all sizes.

Submitted By:	**Verified & Certified By:**
Team ID: Rahul-DS633	Lead Forensic Reviewer
Project: CrimeVision AI	Digital Evidence Admissibility Division
Institution: SmartBridge SkillWallet	Status: **VERIFIED & APPROVED**